Azure Service Bus – Subscription Filters

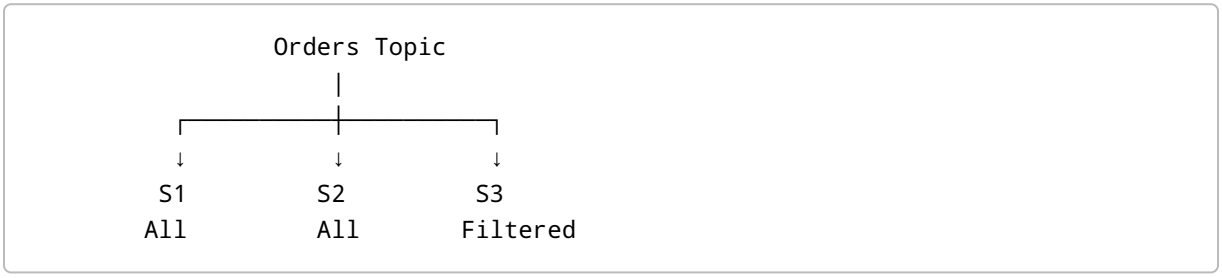
1. What is a Subscription Filter?

In Azure Service Bus, a **topic** can have multiple subscriptions.

Normally, a message published to a topic is delivered to all subscriptions.

But sometimes a subscription does not want to receive every message.

For example:



Suppose:

- S1 wants all messages.
- S2 wants all messages.
- S3 wants only orders where `PaymentStatus = "Completed"`.

In this situation, we use a **subscription filter**.

---

2. Why Do We Need Filters?

Imagine an Orders topic receives different types of orders:

```
Order 1 → PaymentStatus = Completed
Order 2 → PaymentStatus = Pending
Order 3 → PaymentStatus = Failed
Order 4 → PaymentStatus = Completed
```

Suppose the Payment Processing application only wants completed payments.

Without a filter:

```
S3
 ↓
Receives all orders
```

With a filter:

```
S3
 ↓
Only PaymentStatus = Completed
```

So filters allow each subscription to receive only the messages it is interested in.

---

3. Important Concept – Filters Do Not Check the Message Body

This is one of the most important points.

Suppose the message body contains:

```
{
    "OrderId": 1001,
    "PaymentStatus": "Completed"
}
```

The Service Bus subscription filter does **not** directly inspect:

```
Message Body
    ↓
PaymentStatus
```

Instead, we need to put the required information into a **message property**.

---

4. User Properties

When sending a message, we can add custom properties to the message.

For example:

```
message.ApplicationProperties["PaymentStatus"] = "Completed";
```

Now the message contains:

```
Message Body
--------------------------
OrderId = 1001
PaymentStatus = Completed


Application Property
--------------------------
PaymentStatus = Completed
```

The body contains the business data.

The application property provides metadata that can be used by Service Bus filters.

---

5. Why Use Application Properties?

Suppose we have:

```
Order Message

Body:
{
    "OrderId": 1001,
    "PaymentStatus": "Completed"
}
```

Service Bus filters cannot directly evaluate the JSON body.

Therefore, when creating the message, add:

```
message.ApplicationProperties["PaymentStatus"] = "Completed";
```

Now a subscription filter can check:

```
PaymentStatus = 'Completed'
```

This allows Service Bus to decide whether the message should be delivered to that subscription.

---

6. Complete Example

Create a message:

```
var message =
    new ServiceBusMessage(orderJson);
```

Add the application property:

```
message.ApplicationProperties["PaymentStatus"] =
    "Completed";
```

Send the message:

```
await sender.SendMessageAsync(message);
```

The message now has:

```
Body
 ↓
Order details

Application Property
 ↓
PaymentStatus = Completed
```

The subscription filter can use the application property.

---

7. Types of Subscription Filters

Azure Service Bus provides different types of subscription filters.

The important types discussed here are:

1. **Boolean Filter**
2. **SQL Filter**
3. **Correlation Filter**

---

8. Boolean Filter

A Boolean filter is the simplest type of filter.

It can be:

```
TRUE
```

or:

```
FALSE
```

Think of it as:

```
TRUE  → Accept everything
FALSE → Accept nothing
```

TRUE

If the filter evaluates to `true`:

```
Message 1 → Accepted
Message 2 → Accepted
Message 3 → Accepted
```

The subscription receives all messages.

FALSE

If the filter evaluates to `false`:

```
Message 1 → Rejected
Message 2 → Rejected
Message 3 → Rejected
```

The subscription receives no messages.

Easy Way to Remember

```
TRUE  = Catch All
FALSE = Catch Nothing
```

---

9. SQL Filter

A SQL filter allows us to define conditions using a SQL-like expression.

For example:

```
PaymentStatus = 'Completed'
```

This filter checks the message property:

```
PaymentStatus
```

and accepts the message only when its value is:

```
Completed
```

---

10. Example of SQL Filter

Suppose we have these messages:

```
Message 1
PaymentStatus = Completed

Message 2
PaymentStatus = Pending
```

```
Message 3
PaymentStatus = Failed
```

Filter:

```
PaymentStatus = 'Completed'
```

Result:

```
Message 1 → Accepted
Message 2 → Rejected
Message 3 → Rejected
```

Therefore, the subscription receives only Message 1.

---

11. SQL Filter Operators

SQL filters support different types of conditions.

For example:

Equality

```
PaymentStatus = 'Completed'
```

LIKE

```
OrderType LIKE 'Online%'
```

IN

```
PaymentStatus IN ('Completed', 'Approved')
```

This makes SQL filters more flexible than simple equality-based filtering.

---

12. Correlation Filter

A **Correlation Filter** is designed for matching message properties using equality conditions.

For example:

```
PaymentStatus = Completed
```

The filter checks whether the specified property matches the expected value.

Important Point

A correlation filter primarily performs **exact/equality matching**.

It does not provide the broader SQL-style operators available with SQL filters.

---

13. Correlation Filter vs SQL Filter

Both can be used for property-based filtering, but they have different capabilities.

Feature	SQL Filter	Correlation Filter
Equality	Yes	Yes
LIKE	Yes	No
IN	Yes	No
Complex expressions	Yes	No
Property matching	Yes	Yes
Performance	Good	Generally better for simple correlation matching

Simple Example

SQL filter:

```
PaymentStatus = 'Completed'
```

Correlation filter:

```
PaymentStatus = Completed
```

Both can represent a simple equality requirement.

---

14. Why Is Correlation Filter Faster?

Correlation filters are optimized for simple message-property matching.

If your requirement is only:

```
PaymentStatus = Completed
```

a correlation filter is generally more efficient than using a SQL filter for the same simple condition.

Remember

```
Simple equality
      ↓
Correlation Filter

Complex condition
      ↓
SQL Filter
```

---

15. Real-Time Example

Consider an e-commerce system.

There is an:

```
Orders Topic
```

with three subscriptions:

```
Orders Topic
     │
     ├── S1 → All Orders
```

```
      |
      ├── S2 → All Orders
      |
      └── S3 → Completed Payments Only
```

An order message:

```
{
    "OrderId": 1001,
    "PaymentStatus": "Completed"
}
```

is published.

The application also adds:

```
Application Property:
PaymentStatus = Completed
```

Now:

```
S1 → Receives message
S2 → Receives message
S3 → Receives message
```

If another order has:

```
PaymentStatus = Pending
```

then:

```
S1 → Receives message
S2 → Receives message
S3 → Does not receive message
```

---

16. Message Body vs Application Property

This distinction is extremely important for interviews.

Message Body

Contains the actual business data.

Example:

```
{
    "OrderId": 1001,
    "Amount": 5000,
    "PaymentStatus": "Completed"
}
```

Application Property

Contains metadata that can be used for filtering.

Example:

```
PaymentStatus = Completed
```

Conceptually:

```
Service Bus Message
        |
        ├── Body
        |     ↓
        |   Order JSON
        |
        └── Application Properties
              ↓
          PaymentStatus
```

---

17. Why Should We Add the Same Property as the Body?

Suppose the body contains:

```
PaymentStatus = Completed
```

We also add:

```
message.ApplicationProperties["PaymentStatus"] =
    "Completed";
```

Now we have:

```
Body:
PaymentStatus = Completed

Property:
PaymentStatus = Completed
```

The body is used by the application for business processing.

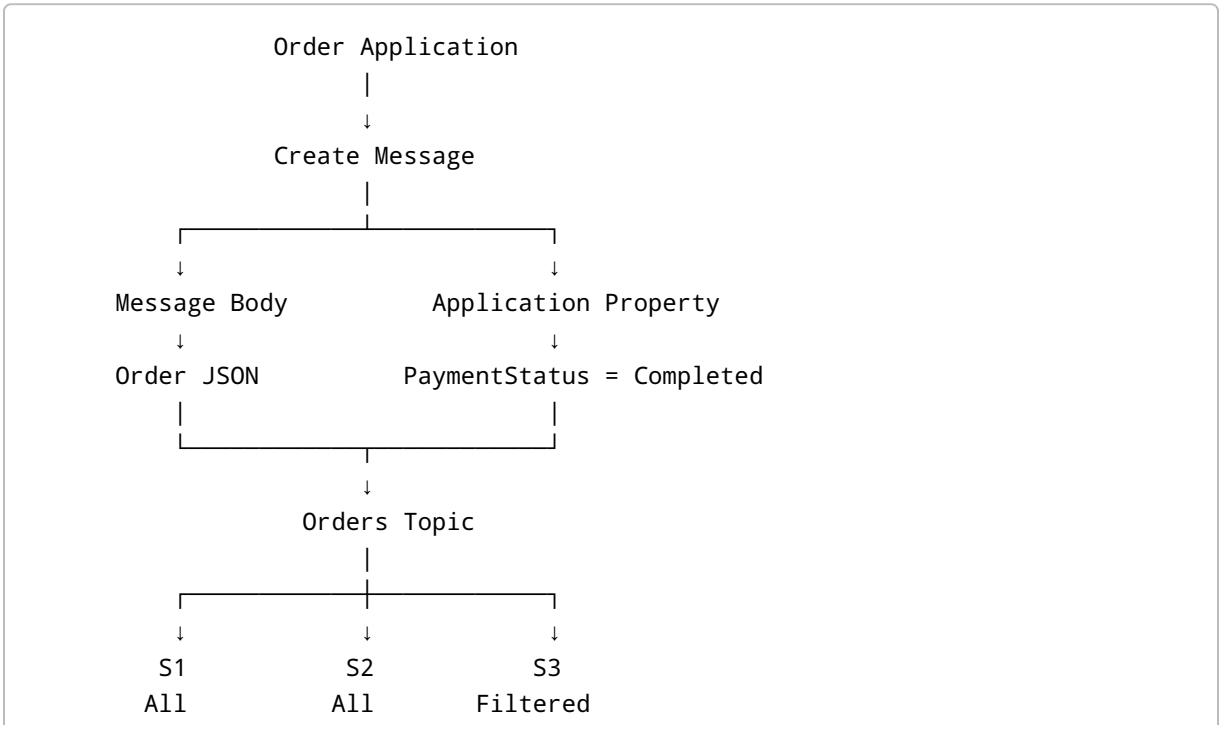
The property is used by Service Bus for subscription filtering.

This avoids requiring the Service Bus filter to understand the application's serialized message body.

---

18. Complete Flow

The complete process looks like this:



```
                                  |
                                  ↓
                    PaymentStatus = Completed
```

---

19. Important Interview Questions

Q1. What is a subscription filter?

Answer:

A subscription filter determines which messages published to a topic should be made available to a particular subscription.

Q2. Does an Azure Service Bus filter inspect the message body?

Answer:

No. Subscription filters operate on message properties/metadata rather than directly evaluating the serialized message body.

If filtering information exists inside the body, we should expose the required value as an application property.

Q3. How can we filter based on PaymentStatus?

Answer:

Add `PaymentStatus` as an application property:

```
message.ApplicationProperties["PaymentStatus"] =
    "Completed";
```

Then create a subscription filter based on that property.

Q4. What are the main types of filters?

Answer:

The commonly used filter types are:

1. Boolean filter
2. SQL filter
3. Correlation filter

Q5. What is a Boolean filter?

Answer:

A Boolean filter accepts either `true` or `false`.

```
TRUE  → Accept all messages
FALSE → Accept no messages
```

Q6. What is a SQL filter?

Answer:

A SQL filter allows SQL-like expressions to evaluate message properties.

For example:

```
PaymentStatus = 'Completed'
```

It supports more expressive conditions such as `LIKE` and `IN`.

Q7. What is a Correlation Filter?

Answer:

A correlation filter is used for efficient matching of message properties, primarily using equality conditions.

For example:

```
PaymentStatus = Completed
```

Q8. Which is better: SQL Filter or Correlation Filter?

Answer:

It depends on the requirement.

For simple equality matching, a **Correlation Filter** is generally preferred because it is optimized for correlation-based matching.

For complex conditions involving operators such as `LIKE`, `IN`, and other SQL-style expressions, use a **SQL Filter**.

---

20. Quick Comparison

```
Boolean Filter
      ↓
TRUE / FALSE
      ↓
All or Nothing


SQL Filter
      ↓
SQL-like conditions
      ↓
Flexible filtering


Correlation Filter
      ↓
Property equality matching
      ↓
Efficient simple filtering
```

---

21. Key Points to Remember

- A topic can have multiple subscriptions.
- Different subscriptions may need different messages.
- **Subscription filters** allow selective message delivery.
- Filters do not directly inspect the serialized message body.
- Use **application properties** for values that need to be filtered.

- Boolean filter:
- `TRUE` → accepts all messages.
- `FALSE` → accepts no messages.
- SQL filter supports SQL-like expressions such as:
- `=`
- `LIKE`
- `IN`
- Correlation filter is primarily for equality-based matching.
- Correlation filters are generally preferred for simple equality matching because they are optimized for that scenario.

Final Memory Trick

```
Need ALL messages?
        ↓
Boolean TRUE


Need NO messages?
        ↓
Boolean FALSE


Need SIMPLE equality?
        ↓
Correlation Filter


Need COMPLEX conditions?
        ↓
SQL Filter


Need to filter based on data inside Body?
        ↓
Put the required value in
Application Properties
        ↓
Create the filter
```

One-Line Interview Answer

> **Azure Service Bus subscription filters allow selective delivery of topic messages. Since filters work with message properties rather than the serialized body, we commonly add relevant values as application properties and then use Boolean, SQL, or Correlation filters to control which messages a subscription receives.**